

Deep Learning

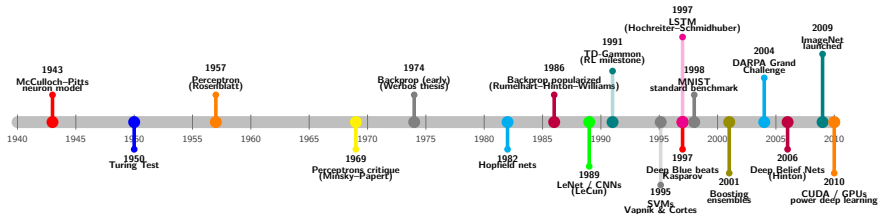
An Introduction

to

Artificial Neural Networks

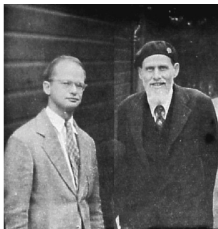
G. Marques

NN an Historical Perspective



1943 Warren McCulloch and Walter Pitts presented a simplified computational model of a biological neuron.

BULLETIN OF
MATHEMATICAL BIOPHYSICS
VOLUME 5, 1943



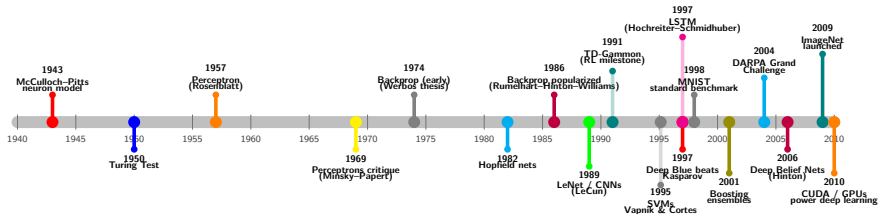
A LOGICAL CALCULUS OF THE IDEAS IMMANENT IN NERVOUS ACTIVITY

WARREN S. MCCULLOCH AND WALTER PITTS

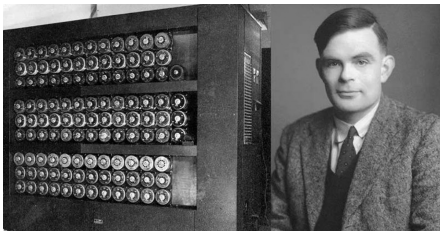
FROM THE UNIVERSITY OF ILLINOIS, COLLEGE OF MEDICINE,
DEPARTMENT OF PSYCHIATRY AT THE ILLINOIS NEUROPSYCHIATRIC INSTITUTE,
AND THE UNIVERSITY OF CHICAGO

Because of the "all-or-none" character of nervous activity, neural events and the relations among them can be treated by means of propositional logic. It is found that the behavior of every net can be described in these terms, with the addition of more complicated logical means for nets containing circles; and that for any logical expression satisfying certain conditions, one can find a net behaving in the fashion it describes.

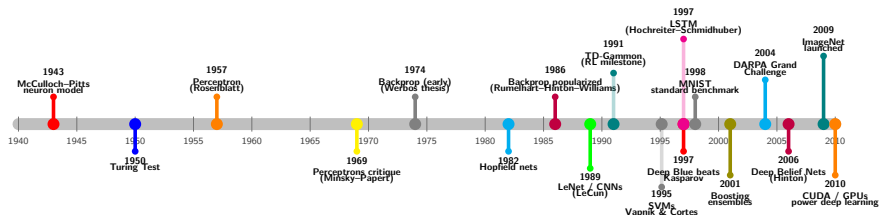
NN an Historical Perspective



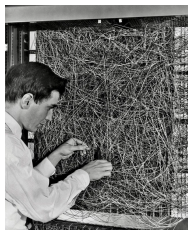
1950 Alan Turing proposed the Turing test where a machine is considered intelligent if a human evaluator cannot reliably distinguish it from a human through conversation.



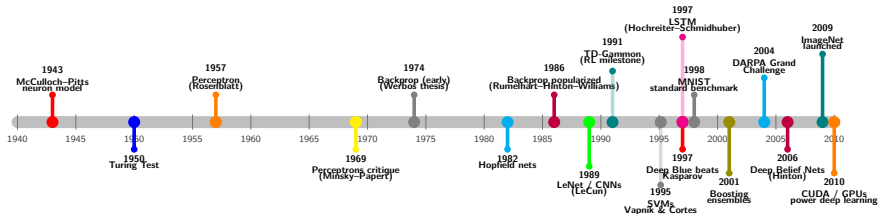
NN an Historical Perspective



1958 Frank Rosenblatt, inspired on the Hebb learning rule and improvements on the McCulloch Pitts model, proposed the perceptron that was able to learn from data.



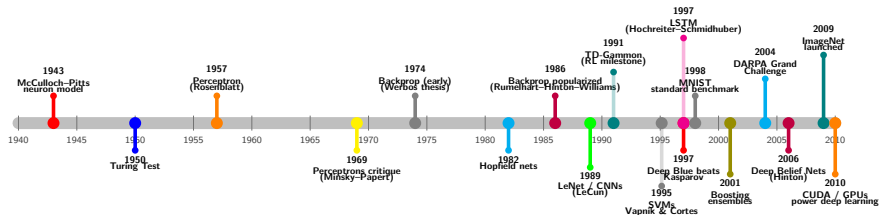
NN an Historical Perspective



1969 Marvin Minsky and Seymour Papert showed the limitations of the perceptron (unable to solve the XOR problem), and contributed to the “AI winter”.

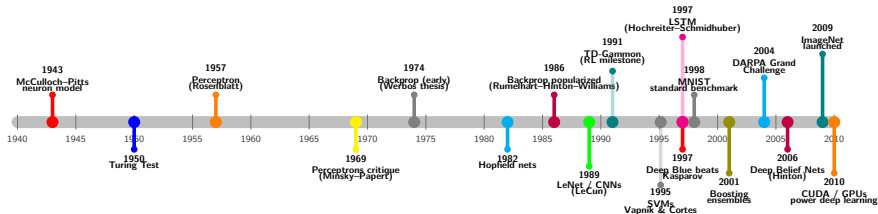


NN an Historical Perspective



1970-80 1st "AI winter".

NN an Historical Perspective



1986 David Rumelhart, Geoffrey Hinton and Ronald Williams proposed the Multi-Layer Perceptron (MLPs) and the back propagation learning rule (this rule had already been proposed by Paul Werbos (1974) and others). This marked the end of the “AI winter”.

Learning representations by back-propagating errors

David E. Rumelhart*, Geoffrey E. Hinton† & Ronald J. Williams*

* Institute for Cognitive Science, C-015, University of California, San Diego, La Jolla, California 92093, USA

† Department of Computer Science, Carnegie-Mellon University, Pittsburgh, Philadelphia 15213, USA



We describe a new learning procedure, back-propagation, for networks of neurone-like units. The procedure repeatedly adjusts the weights of the connections in the network so as to minimize a measure of the difference between the actual output vector of the net and the desired output vector. As a result of the weight adjustments, internal ‘hidden’ units which are not part of the input or output come to represent important features of the task domain, and the regularities in the task are captured by the interactions of these units. The ability to create useful new features distinguishes back-propagation from earlier, simpler methods such as the perceptron-convergence procedure¹.

There have been many attempts to design self-organizing neural networks. The aim is to find a powerful synaptic modification rule that will allow an arbitrarily connected neural network to develop an internal structure that is appropriate for a particular task domain. The task is specified by giving the desired state vector of the output units for each state vector of

more difficult when we introduce hidden units whose actual or desired states are not specified by the task. (In perceptrons, there are ‘feature analyzers’ between the input and output that are not true hidden units because their input connections are fixed by hand, so their states are completely determined by the input vector: they do not learn representations.) The learning procedure must decide under what circumstances the hidden units should be active in order to help achieve the desired input-output behaviour. This amounts to deciding what these units should represent. We demonstrate that a general purpose and relatively simple procedure is powerful enough to construct appropriate internal representations.

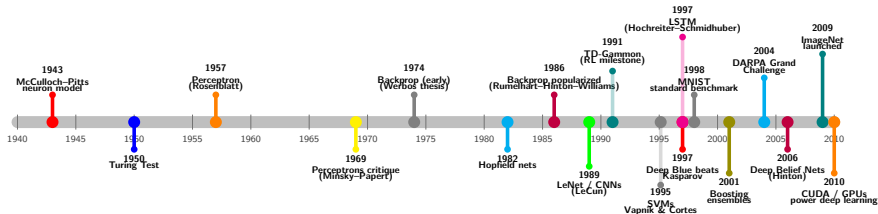
The simplest form of the learning procedure is for layered networks which have a layer of input units at the bottom; any number of intermediate layers; and a layer of output units at the top. Connections within a layer or from higher to lower layers are forbidden, but connections can skip intermediate layers. An input vector is presented to the network by setting the states of the input units. Then the states of the units in each layer are determined by applying equations (1) and (2) to the connections coming from lower layers. All units within a layer have their states set sequentially, starting at the bottom and working upwards until the states of the output units are determined.

The total input, x_i , to unit j is a linear function of the outputs, y_i , of the units that are connected to j and of the weights, w_{ji} , on these connections

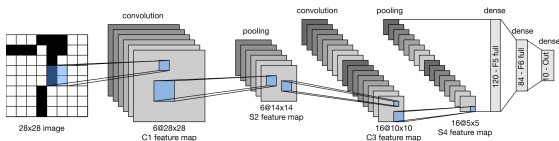
$$x_j = \sum_i w_{ji} y_i \quad (1)$$

Units can be given biases by introducing an extra input to each

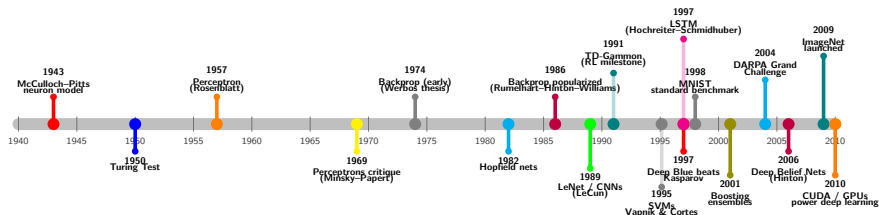
NN an Historical Perspective



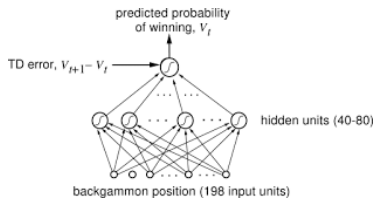
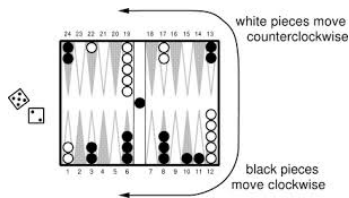
1989 Yann LeCun introduced early convolutional neural networks (CNNs), notably LeNet, designed for handwritten digit recognition. These networks used convolution and pooling layers and were later applied to tasks such as bank check and ZIP-code recognition.



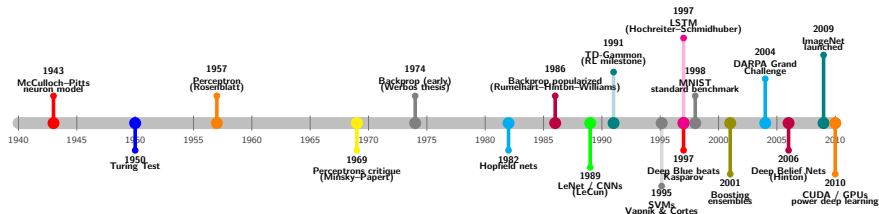
NN an Historical Perspective



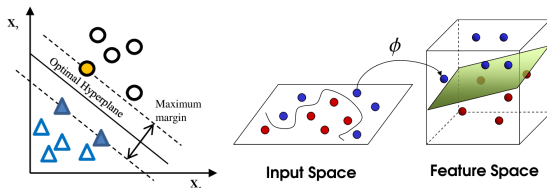
1991 Developed by Gerald Tesauro, TD-Gammon used reinforcement learning and self-play to learn high-level backgammon strategies. It demonstrated that neural networks could learn complex decision-making without explicit programming.



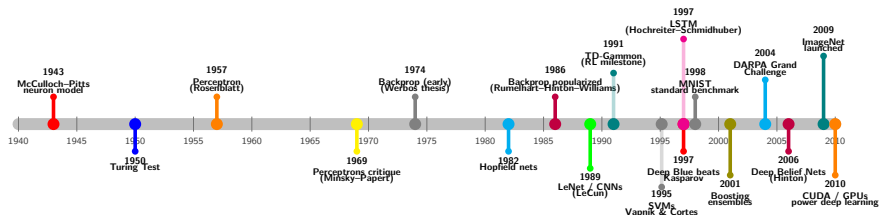
NN an Historical Perspective



1995 Vapnik and Cortes introduce Support Vector Machines, which become the leading classification method in machine learning before the deep learning resurgence.



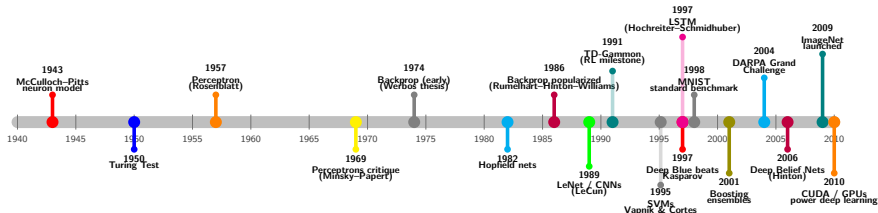
NN an Historical Perspective



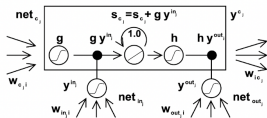
1990s-12 2nd “AI winter”.

Between 1990 and 2012, neural networks remained an active research area but were largely overshadowed by statistical machine learning methods.

NN an Historical Perspective



1997 Sepp Hochreiter and Jürgen Schmidhuber introduced Long Short-Term Memory (LSTM) networks, a type of recurrent neural network designed to overcome the vanishing gradient problem and capture long-range dependencies in sequences.

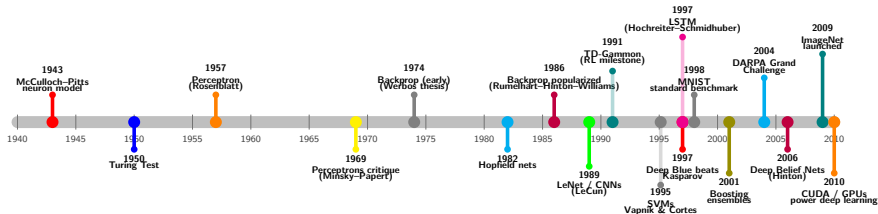


S. Hochreiter



J. Schmidhuber

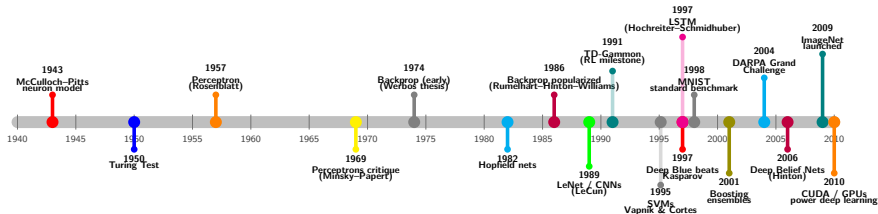
NN an Historical Perspective



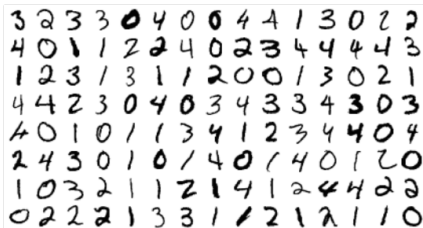
1997 Deep Blue defeated world chess champion Garry Kasparov. Although not based on learning neural networks, the system demonstrated the power of massive computation and search in AI.



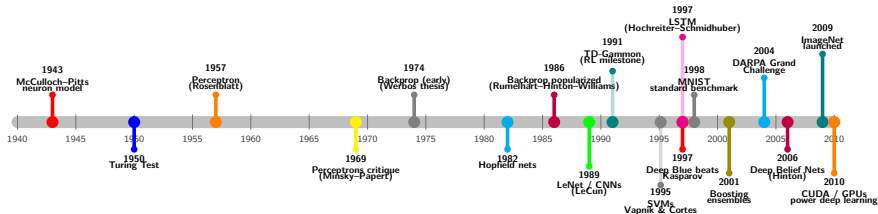
NN an Historical Perspective



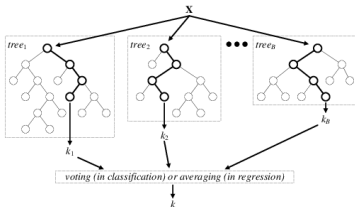
1998 The MNIST dataset of handwritten digits was introduced by Yann LeCun and collaborators. It became the standard benchmark dataset for machine learning, especially for testing neural network models.



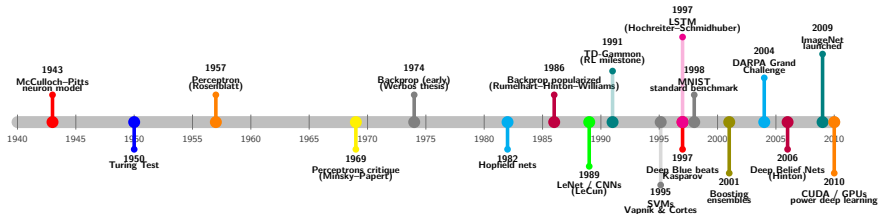
NN an Historical Perspective



2001 Techniques such as AdaBoost and ensemble methods became widely adopted in machine learning. Combining multiple weak models into a stronger predictor significantly improved classification performance in many applications.



NN an Historical Perspective

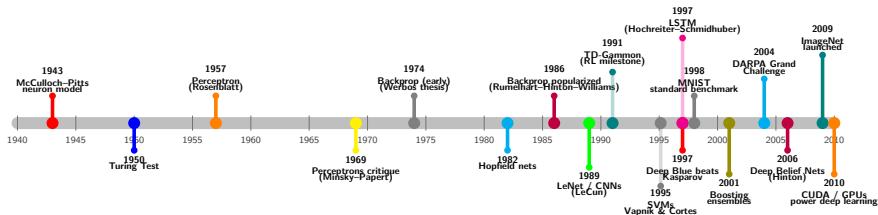


2004 1st DARPA Grand Challenge.

- ▶ DARPA: Defense Advanced Research Projects Agency.
- ▶ Vehicles had to navigate a ≈ 240 km route in the Mojave Desert.
- ▶ No human intervention was allowed during the race.
- ▶ **Prize:** \$1 million.
- ▶ **Outcome:** No team finished; the best vehicle traveled about **12 km**.



NN an Historical Perspective

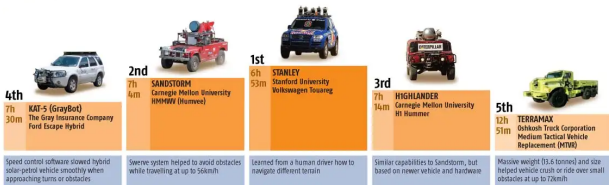


2005 2nd DARPA Grand Challenge.

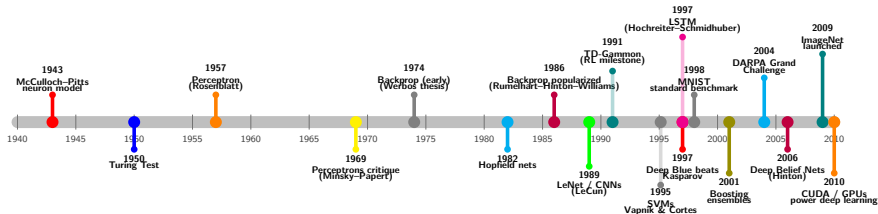
- ▶ Second autonomous vehicle competition in the Mojave Desert.
- ▶ Vehicles had to complete a $\approx 212\text{km}$ off-road route.
- ▶ **Prize: \$2 million.**
- ▶ **Five vehicles successfully completed the course.**
- ▶ Winner: Team from Stanford University with the vehicle **Stanley**.

AND THE WINNERS ARE...

Of the 23 entrants in this year's DARPA Grand Challenge robot vehicle race, five reached the end of the 132km course, outside Primm, Nevada. In last year's race, no one got further than 11km from the start



NN an Historical Perspective



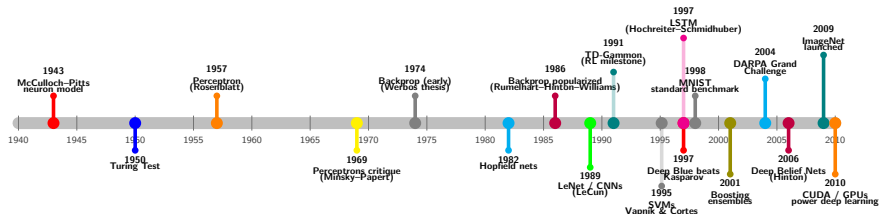
2007 DARPA Urban Challenge.

Autonomous vehicles had to navigate a mock city while obeying traffic rules, merging with traffic, handling intersections and avoiding other vehicles.

Winner: Team Tartan Racing from Carnegie Mellon University with the vehicle Boss.

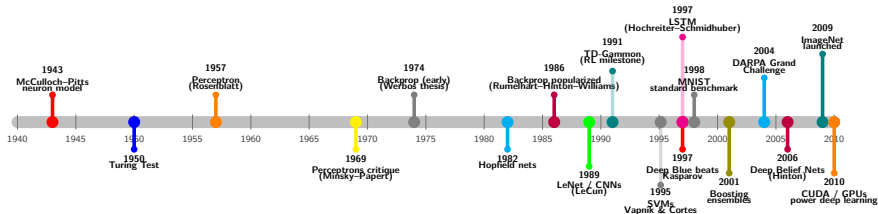


NN an Historical Perspective



2006 Deep Belief Networks (DBNs) were introduced in 2006 by Geoffrey Hinton and collaborators as a method to train deep neural networks more effectively. DBNs are composed of multiple layers of Restricted Boltzmann Machines (RBMs) trained one layer at a time using unsupervised learning. After this layer-wise pretraining, the whole network can be fine-tuned with supervised learning.

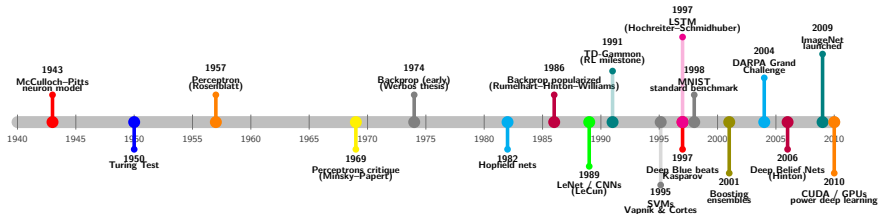
NN an Historical Perspective



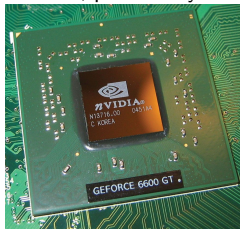
2009 The ImageNet Large Scale Visual Recognition Challenge (2009-2017) was a major computer vision competition based on the ImageNet, created by Fei-Fei Li and collaborators. Participants developed algorithms to classify and detect objects in millions of labeled images across thousands of categories.



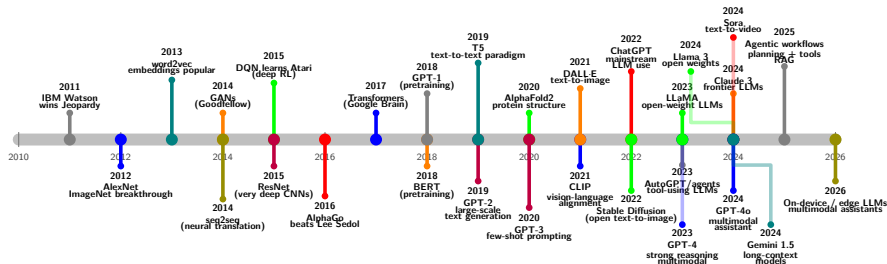
NN an Historical Perspective



- 2010** Researchers began using graphics processing units (GPUs) to accelerate the training of neural networks. GPUs are highly efficient for parallel numerical computations, especially the matrix operations used in deep learning. The widespread adoption of GPUs was largely driven by hardware and software from NVIDIA, particularly the CUDA programming platform introduced in 2007.

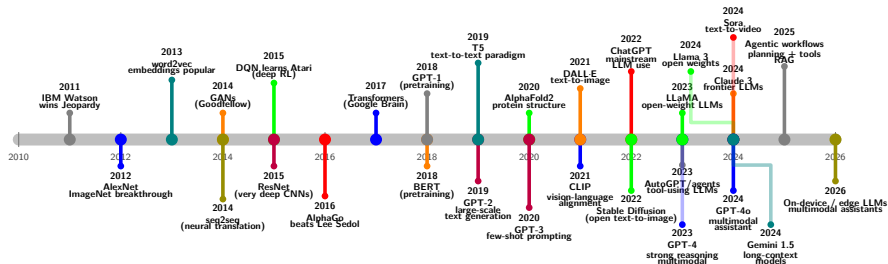


NN an Historical Perspective



2011 Watson wins Jeopardy:
IBM Watson defeated human champions on the quiz show Jeopardy!, demonstrating major progress in natural language processing and question-answering systems.

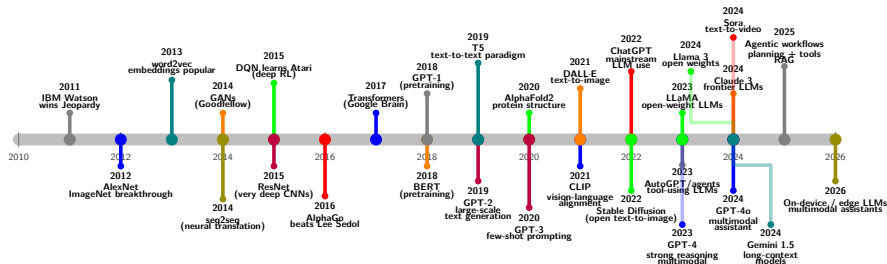
NN an Historical Perspective



2012 AlexNet

AlexNet, developed by Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton at the University of Toronto, achieved a dramatic improvement in the ImageNet Large Scale Visual Recognition Challenge. Trained using GPUs, it demonstrated the power of deep convolutional neural networks.

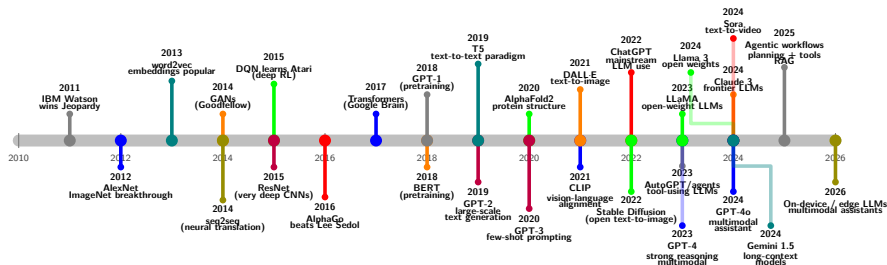
NN an Historical Perspective



2013 word2vec

word2vec was developed by Tomas Mikolov and colleagues at Google. The method learned vector representations of words from large corpora, capturing semantic relationships such as analogies.

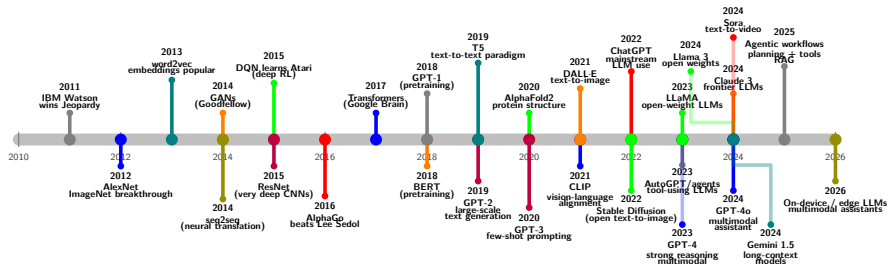
NN an Historical Perspective



2014 Sequence-to-sequence models.

The sequence-to-sequence architecture for neural machine translation was introduced by Ilya Sutskever, Oriol Vinyals, and Quoc V. Le at Google. It enabled neural networks to map input sequences to output sequences, transforming machine translation.

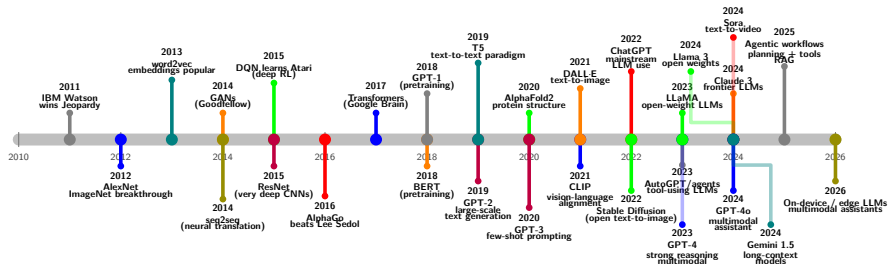
NN an Historical Perspective



2014 GANs

Generative Adversarial Networks were introduced by Ian Goodfellow and collaborators while at the University of Montreal. GANs use two competing neural networks (generator and discriminator) to produce realistic synthetic data.

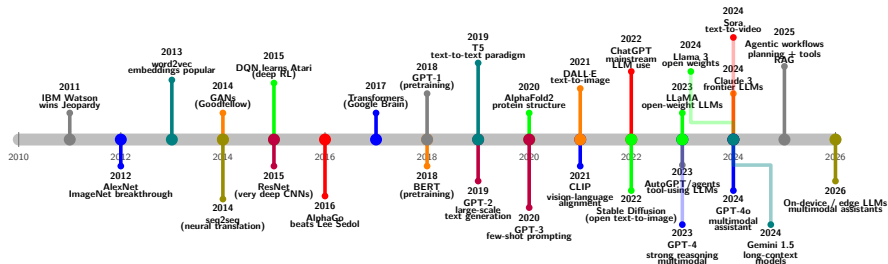
NN an Historical Perspective



2015 ResNet

Residual Network was developed by Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun at Microsoft. Residual connections allowed training extremely deep networks and became a standard design in deep learning.

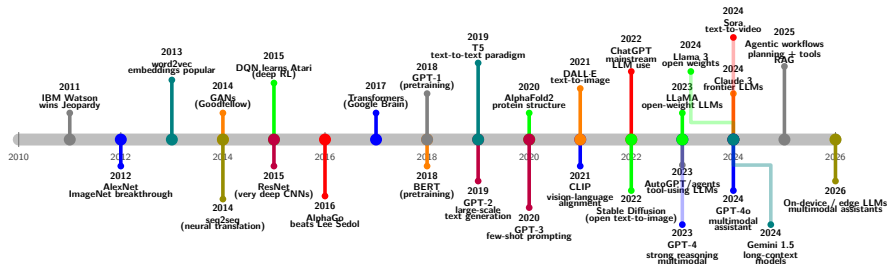
NN an Historical Perspective



2015 Deep Q-Networks

Researchers at DeepMind including Volodymyr Mnih demonstrated that reinforcement learning combined with deep neural networks could learn to play Atari games directly from raw pixels.

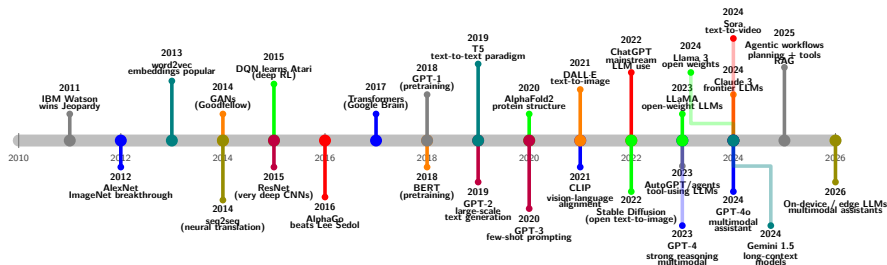
NN an Historical Perspective



2016 AlphaGo

AlphaGo developed by DeepMind defeated world champion Lee Sedol. The system combined deep neural networks with reinforcement learning and self-play, marking a major milestone in AI.

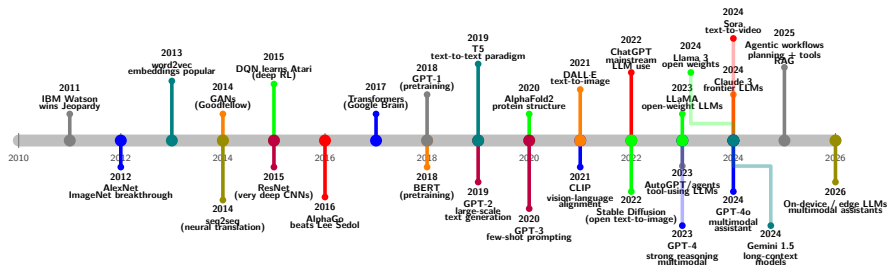
NN an Historical Perspective



2017 Transformer architecture

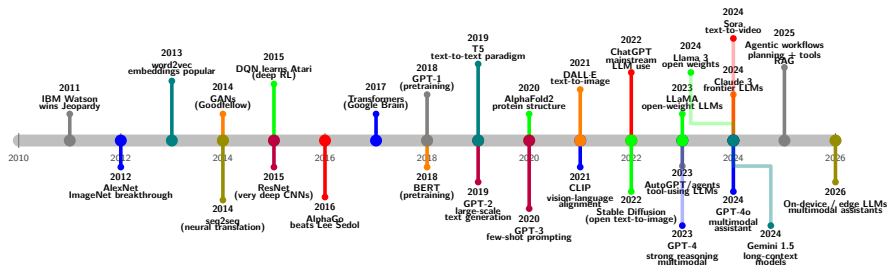
The Transformer model was introduced in the paper Attention Is All You Need by researchers including Ashish Vaswani and Noam Shazeer at Google. Transformers rely on self-attention mechanisms rather than recurrent networks, enabling large-scale language models.

NN an Historical Perspective



2018 BERT
BERT (Bidirectional Encoder Representations from Transformers) introduced bidirectional contextual representations for language, significantly improving many NLP tasks.

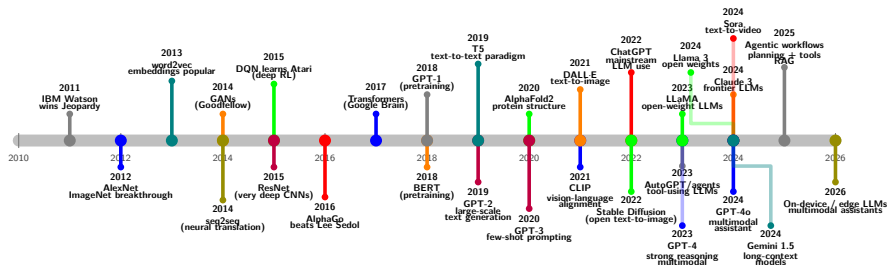
NN an Historical Perspective



2018 GPT-1

The first Generative Pre-trained Transformer (GPT) model was developed by researchers including Alec Radford at OpenAI. It demonstrated that large language models pretrained on large corpora could be adapted to multiple tasks.

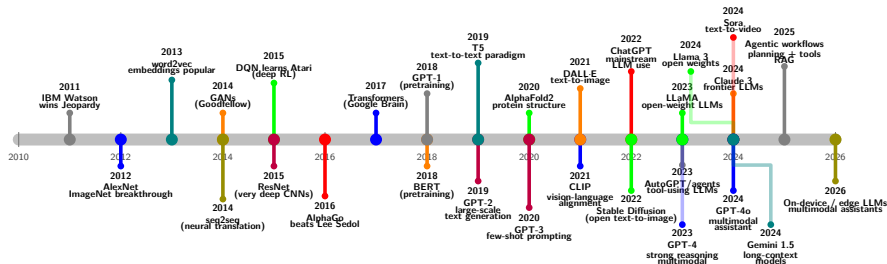
NN an Historical Perspective



2019 GPT-2

GPT-2 from OpenAI showed that scaling language models significantly improved text generation quality and coherence, raising attention to generative AI.

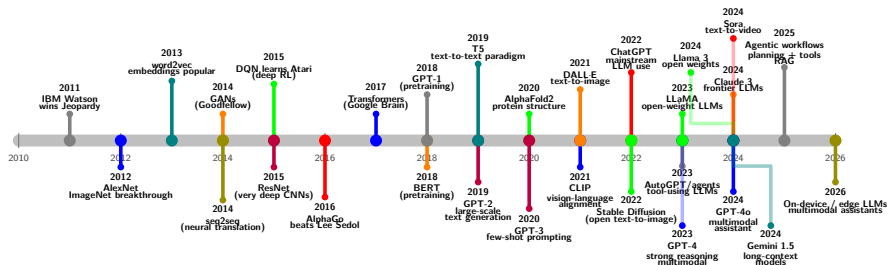
NN an Historical Perspective



2019 T5

The Text-to-Text Transfer Transformer (T5) model was developed by researchers at Google including Colin Raffel. It unified many NLP tasks into a single text-to-text framework.

NN an Historical Perspective



2019 GPT-3

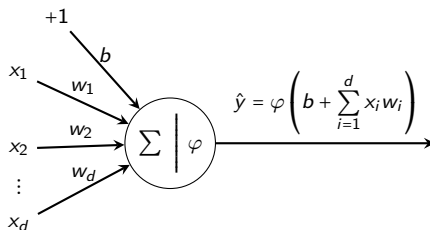
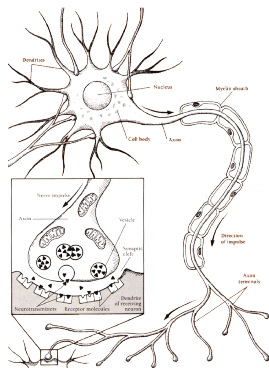
GPT-3 from OpenAI contained 175 billion parameters and demonstrated strong few-shot learning capabilities.

NN an Historical Perspective

Why Deep Learning? Why Now?

- Hardware
CPUs, GPUs and TPUs, and computer clusters.
- Data
Internet allows the collections and distribution of large (huge) datasets.
- Algorithmic Advances
Better initializations, activations and optimizations
- Deep Learning automates the step of *feature engineering*.

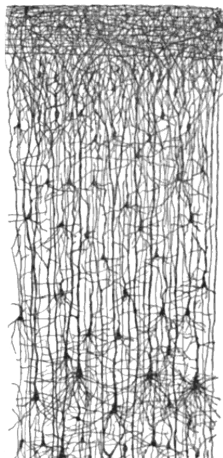
The Perceptron



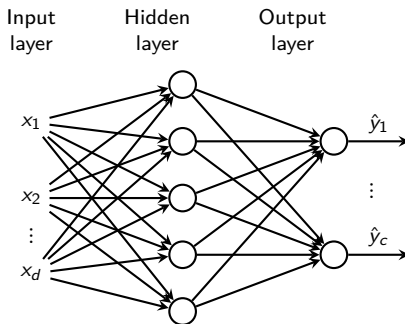
- In the perceptron, the activation function is a step function.
- In log-linear classifiers, φ is either a sigmoid or hyperbolic tangent.
- In neural networks, φ can also be the previous two functions, and also many others (not biologically inspired).

NOTE: The perceptron and log-linear classifiers have several limitations. These are overcome by multi-layer perceptrons, which are basically stacked layers of log-linear classifiers (also inspired by the connections between neurons in the brain).

Multi-Layer Perceptron



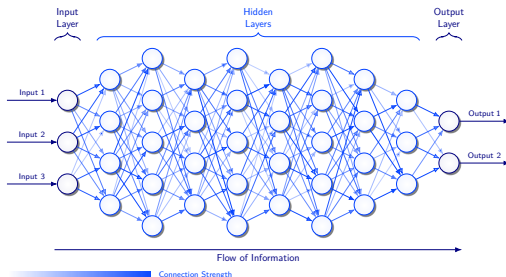
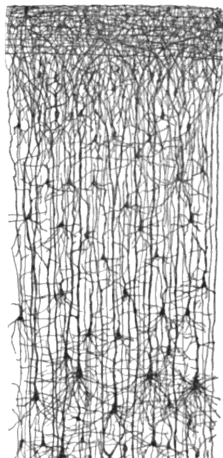
- Simple, three layer MLP networks:



- Each circle represents a non-linear, perceptron-like operation.
- Such networks can solve the XOR problem and other non-linear ones.

Multi-Layer Perceptron

- Deep neural network:



- Capable of solving highly complex problems.

Training MLPs

Backpropagation Algorithm

- A MLP network consists of one input layer, one or more hidden layers, and output layer. In classification the number of outputs, $\hat{y}_i$ corresponds to the number of classes.

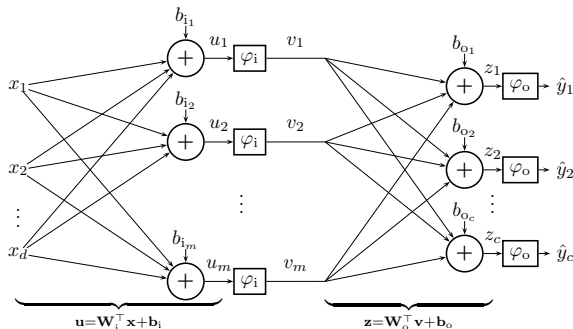
ALGORITHM:

- **FORWARD PASS:**
Each training instance, $\mathbf{x}[n]$ is fed through the network to produce an output $\hat{\mathbf{y}}[n]$.
- **BACKWARD PASS:**
The error for each instance is computed and the gradient is fed back through the network. A typical error function is the squared difference between the output and desired response for that instance: $\mathbf{e}[n] = \|\mathbf{y}[n] - \hat{\mathbf{y}}[n]\| = \sum_i (y_i - \hat{y}_i)^2$
It is necessary to calculate the gradient for all the weights of each neuron in the network and adapt them through gradient descent.

Training MLPs

Backpropagation Algorithm

EXAMPLE: MLP network with d inputs, m hidden units and c outputs.



- Error function: Mean Squared Error (MSE) $\mathcal{E} = \frac{1}{N} \sum_{n=1}^N \|\mathbf{y}_n - \hat{\mathbf{y}}_n\|^2 = \frac{1}{N} \sum_{n=1}^N \|\mathbf{e}_n\|^2$
- Weight adaptation through steepest gradient descent (i.e. calculate $\frac{\partial \mathcal{E}}{\partial \mathbf{W}_i}, \frac{\partial \mathcal{E}}{\partial \mathbf{b}_i}, \frac{\partial \mathcal{E}}{\partial \mathbf{W}_o}, \frac{\partial \mathcal{E}}{\partial \mathbf{b}_o}$).

Training MLPs

Backpropagation Algorithm

EXAMPLE: MLP network with d inputs, m hidden units and c outputs.

- Weights and parameter dimensions:
 - ▶ $\mathbf{y}$, $\hat{\mathbf{y}}$, $\mathbf{z}$ and $\mathbf{b}_o$ are c dimensional vectors
 - ▶ $\mathbf{v}$, $\mathbf{u}$ and $\mathbf{b}_i$ are m dimensional vectors
 - ▶ input training vector $\mathbf{x}$ is a d dimensional vector
 - ▶ $\mathbf{W}_i$ is a $d \times m$ dimensional matrix
 - ▶ $\mathbf{W}_o$ is a $m \times c$ dimensional matrix

Gradients for output biases $\mathbf{b}_o$:

$$\frac{\partial \|\mathbf{e}\|^2}{\partial \mathbf{b}_o} = \left[\frac{\partial \|\mathbf{e}\|^2}{\partial b_{o_1}}, \dots, \frac{\partial \|\mathbf{e}\|^2}{\partial b_{o_c}} \right]^T$$

$$\begin{aligned} \frac{\partial \|\mathbf{e}\|^2}{\partial b_{o_j}} &= \frac{\partial \|\mathbf{e}\|^2}{\partial \hat{y}_j} \frac{\partial \hat{y}_j}{\partial z_j} \frac{\partial z_j}{\partial b_{o_j}} \\ &= -2(y_j - \hat{y}_j) \varphi'_o(z_j) \end{aligned}$$

- ▶ $\frac{\partial \|\mathbf{e}\|^2}{\partial \hat{y}_j} = -2e_j = -2(y_j - \hat{y}_j)$
- ▶ $\frac{\partial \hat{y}_j}{\partial z_j} = \varphi'_o(z_j) = \begin{cases} \hat{y}_j - \hat{y}_j^2 & \text{for sigmoids} \\ 1 - \hat{y}_j^2 & \text{for hyperbolic tangents} \end{cases}$
- ▶ $\frac{\partial z_j}{\partial b_{o_j}} = 1$

Training MLPs

Backpropagation Algorithm

EXAMPLE: MLP network with d inputs, m hidden units and c outputs.

- Weights and parameter dimensions:
 - ▶ $\mathbf{y}$, $\hat{\mathbf{y}}$, $\mathbf{z}$ and $\mathbf{b}_o$ are c dimensional vectors
 - ▶ $\mathbf{v}$, $\mathbf{u}$ and $\mathbf{b}_i$ are m dimensional vectors
 - ▶ input training vector $\mathbf{x}$ is a d dimensional vector
 - ▶ $\mathbf{W}_i$ is a $d \times m$ dimensional matrix
 - ▶ $\mathbf{W}_o$ is a $m \times c$ dimensional matrix

Gradients for output weights $\mathbf{W}_o$.

$\frac{\partial \|\mathbf{e}\|^2}{\partial \mathbf{W}_o}$ is a $m \times c$ matrix.

$$\begin{aligned}\frac{\partial \|\mathbf{e}\|^2}{\partial w_{oij}} &= \frac{\partial \|\mathbf{e}\|^2}{\partial \hat{y}_j} \frac{\partial \hat{y}_j}{\partial z_j} \frac{\partial z_j}{\partial w_{oij}} \\ &= -2(y_j - \hat{y}_j) \varphi'_o(z_j) v_i\end{aligned}$$

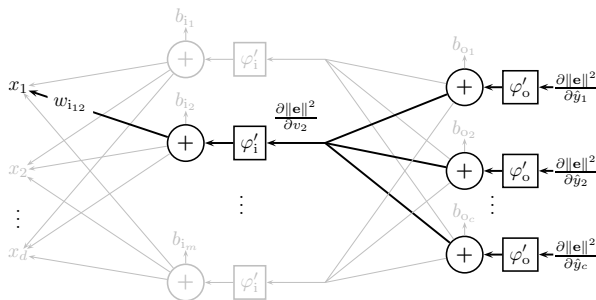
- ▶ $\frac{\partial \|\mathbf{e}\|^2}{\partial \hat{y}_j} = -2e_j = -2(y_j - \hat{y}_j)$
- ▶ $\frac{\partial \hat{y}_j}{\partial z_j} = \varphi'_o(z_j) = \begin{cases} \hat{y}_j - \hat{y}_j^2 & \text{for sigmoids} \\ 1 - \hat{y}_j^2 & \text{for hyperbolic tangents} \end{cases}$
- ▶ $\frac{\partial z_j}{\partial w_{oij}} = v_i$

Training MLPs

Backpropagation Algorithm

EXAMPLE: MLP network with d inputs, m hidden units and c outputs.

Gradients for input weights $\mathbf{W}_i$ and $\mathbf{b}_i$:



- All the paths from the output to the parameter to adapt, w_{12} , must be considered:

- $$\frac{\partial \|\mathbf{e}\|^2}{w_{i12}} = \sum_{k=1}^c \frac{\partial \|\mathbf{e}\|^2}{\partial \hat{y}_k} \frac{\partial \hat{y}_k}{\partial z_k} \frac{\partial z_k}{\partial v_2} \frac{\partial v_2}{\partial u_2} \frac{\partial u_2}{\partial w_{i12}}$$

Training MLPs

Backpropagation Algorithm

EXAMPLE: MLP network with d inputs, m hidden units and c outputs.

Gradients for input biases $\mathbf{b}_i$:

$$\begin{aligned}\frac{\partial \|\mathbf{e}\|^2}{\partial \mathbf{b}_i} &= \left[\frac{\partial \|\mathbf{e}\|^2}{\partial b_{i1}}, \dots, \frac{\partial \|\mathbf{e}\|^2}{\partial b_{im}} \right]^T \\ \frac{\partial \|\mathbf{e}\|^2}{\partial b_{ij}} &= -2 \frac{\partial v_j}{\partial u_j} \frac{\partial u_j}{\partial b_{ij}} \sum_{k=1}^c e_k \varphi'_o(z_k) \frac{\partial z_k}{\partial v_j} \\ &= -2 \varphi'_i(u_j) \sum_{k=1}^c w_{ojk} e_k \varphi'_o(z_k)\end{aligned}$$

Gradients for input weights $\mathbf{W}_j$:

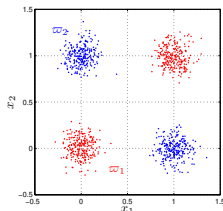
$$\begin{aligned}\frac{\partial \|\mathbf{e}\|^2}{\partial w_{ij}} &= -2 x_i \frac{\partial v_j}{\partial u_j} \frac{\partial u_j}{\partial w_{ij}} \sum_{k=1}^c e_k \varphi'_o(z_k) \frac{\partial z_k}{\partial v_j} \\ &= -2 x_i \varphi'_i(u_j) \sum_{k=1}^c w_{ojk} e_k \varphi'_o(z_k)\end{aligned}$$

- ▶ $\frac{\partial \|\mathbf{e}\|^2}{\partial \hat{y}_k} = -2e_k = -2(y_k - \hat{y}_k)$
- ▶ $\frac{\partial \hat{y}_k}{\partial z_k} = \varphi'_o(z_k) = \begin{cases} \hat{y}_k - \hat{y}_k^2 & \text{sigmoids} \\ 1 - \hat{y}_k^2 & \text{hyper. tang.} \end{cases}$
- ▶ $\frac{\partial z_k}{\partial v_j} = w_{ojk}$
- ▶ $\frac{\partial v_j}{\partial u_j} = \varphi'_i(u_j) = \begin{cases} v_j - v_j^2 & \text{sigmoids} \\ 1 - v_j^2 & \text{hyper. tang.} \end{cases}$
- ▶ $\frac{\partial u_j}{\partial b_{ij}} = 1$
- ▶ $\frac{\partial u_j}{\partial w_{ij}} = x_i$

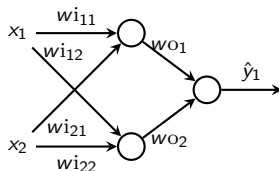
Training MLPs

Example: XOR problem

Binary classification problem (2D data with 500 pts/class)



Consider the following NN with activation functions $\varphi(x) = \tanh(x)$.



NOTE: There is also the biases b_1 and b_2 of the hidden nodes (admit that the output node has no bias).

- Output of the first hidden node: $v_1 = \tanh(w_{i11}x_1 + w_{i21}x_2 + b_1)$
- Output of the second hidden node: $v_2 = \tanh(w_{i12}x_1 + w_{i22}x_2 + b_2)$
- Output of the network: $\hat{y} = \tanh(v_1w_{o1} + v_2w_{o2})$

Objective: Write a Python program that adapts the parameters of the network by minimizing the MSE through steepest gradient descent.

Training MLPs

Example: XOR problem

Binary classification problem (2D data with 500 pts/class)

PROGRAMMING CONSIDERATIONS:

1. Initializations: weights (e.g Gaussian), adaptation step η , number of iterations.
 - ▶ Calculate error
2. Adaptation - for the whole training set :
 - ▶ Calculate gradients
 - ▶ Adapt weights
3. Repeat until maximum number of iterations is reached.

Training MLPs

Example: XOR problem

Binary classification problem (2D data with 500 pts/class)

- MSE function: $\mathcal{E} = \frac{1}{N} \sum_{n=0}^{N-1} (y[n] - \hat{y}[n])^2 = \frac{1}{N} \sum_{n=0}^{N-1} (y[n] - \tanh(w_{o1} v_1[n] + w_{o2} v_2[n]))^2$

- Network operations:

- ▶ Hidden layer:

$$\begin{aligned} u_1 &= w_{i11} x_1 + w_{i21} x_2 + b_1 & u_2 &= w_{i12} x_1 + w_{i22} x_2 + b_1 \\ v_1 &= \tanh(w_{i11} x_1 + w_{i21} x_2 + b_1) & v_2 &= \tanh(w_{i12} x_1 + w_{i22} x_2 + b_1) \end{aligned}$$

- ▶ Output:

$$\begin{aligned} z &= w_{o1} v_1 + w_{o2} v_2 \\ \hat{y} &= \tanh(z) = \tanh(w_{o1} v_1 + w_{o2} v_2) \\ &= \tanh(w_{o1} \tanh(w_{i11} x_1 + w_{i21} x_2 + b_1) + w_{o2} \tanh(w_{i12} x_1 + w_{i22} x_2 + b_1)) \end{aligned}$$

- Matrix notation:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \end{bmatrix}, \mathbf{W}_i = \begin{bmatrix} w_{i11} & w_{i12} \\ w_{i21} & w_{i22} \end{bmatrix}, \mathbf{u} = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} = \mathbf{W}_i^T \mathbf{x} + \mathbf{b}, \mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \tanh(\mathbf{u}), \mathbf{w}_o = \begin{bmatrix} w_{o1} \\ w_{o2} \end{bmatrix}$$

$$z = \mathbf{w}_o^T \mathbf{v}$$

$$\hat{y} = \tanh(\mathbf{w}_o^T \mathbf{v}) = \tanh(\mathbf{w}_o^T \tanh(\mathbf{W}_i^T \mathbf{x} + \mathbf{b}))$$

Training MLPs

Example: XOR problem

Binary classification problem (2D data with 500 pts/class)

- Gradients of output weights $\mathbf{w}_o$:

$$\frac{\partial \mathcal{E}}{\partial w_{o1}} = \frac{-2}{N} \sum_{n=0}^{N-1} (y[n] - \hat{y}[n]) \frac{\partial \hat{y}[n]}{\partial z[n]} \frac{\partial z[n]}{\partial w_{o1}} = \frac{-2}{N} \sum_{n=0}^{N-1} e[n] (1 - \hat{y}[n]^2) v_1[n]$$

$$\frac{\partial \mathcal{E}}{\partial w_{o2}} = \frac{-2}{N} \sum_{n=0}^{N-1} (y[n] - \hat{y}[n]) \frac{\partial \hat{y}[n]}{\partial z[n]} \frac{\partial z[n]}{\partial w_{o2}} = \frac{-2}{N} \sum_{n=0}^{N-1} e[n] (1 - \hat{y}[n]^2) v_2[n]$$

Training MLPs

Example: XOR problem

Binary classification problem (2D data with 500 pts/class)

- Gradient for output weights $\frac{\partial \mathcal{E}}{\partial \mathbf{w}_o} = -\frac{2}{N} \sum_n e[n] (1 - \hat{y}[n]^2) \mathbf{v}[n]$

- Gradient for input biases $\mathbf{b}$:

$$\frac{\partial \mathcal{E}}{\partial b_1} = \frac{-2}{N} \sum_{n=0}^{N-1} (y[n] - \hat{y}[n]) \frac{\partial \hat{y}[n]}{\partial z[n]} \frac{\partial z[n]}{\partial v_1[n]} \frac{\partial v_1[n]}{\partial u_1[n]} \frac{\partial u_1[n]}{\partial b_1}$$

$$= \frac{-2}{N} \sum_{n=0}^{N-1} e[n] (1 - \hat{y}[n]^2) w_{o1} (1 - v_1[n]^2) \times 1$$

$$\frac{\partial \mathcal{E}}{\partial b_2} = \frac{-2}{N} \sum_{n=0}^{N-1} (y[n] - \hat{y}[n]) \frac{\partial \hat{y}[n]}{\partial z[n]} \frac{\partial z[n]}{\partial v_2[n]} \frac{\partial v_2[n]}{\partial u_2[n]} \frac{\partial u_2[n]}{\partial b_2}$$

$$= \frac{-2}{N} \sum_{n=0}^{N-1} e[n] (1 - \hat{y}[n]^2) w_{o2} (1 - v_2[n]^2) \times 1$$

Training MLPs

Example: XOR problem

Binary classification problem (2D data with 500 pts/class)

- Gradient for output weights $\frac{\partial \mathcal{E}}{\partial \mathbf{w}_o} = -\frac{2}{N} \sum_n e[n] (1 - \hat{y}[n]^2) \mathbf{v}[n]$
- Gradient for input biases $\frac{\partial \mathcal{E}}{\partial \mathbf{b}} = -\frac{2}{N} \sum_n e[n] (1 - \hat{y}[n]^2) \mathbf{w}_o (1 - \mathbf{v}[n]^2)$
- Gradient for input weights $\mathbf{W}_i$:

$$\begin{aligned} \frac{\partial \mathcal{E}}{\partial w_{i11}} &= \frac{-2}{N} \sum_{n=0}^{N-1} (y[n] - \hat{y}[n]) \frac{\partial \hat{y}[n]}{\partial z[n]} \frac{\partial z[n]}{\partial v_1[n]} \frac{\partial v_1[n]}{\partial u_1[n]} \frac{\partial u_1[n]}{\partial w_{i11}} \\ &= \frac{-2}{N} \sum_{n=0}^{N-1} e[n] (1 - \hat{y}[n]^2) w_{o1} (1 - v_1[n]^2) x_1[n] \end{aligned}$$

$$\begin{aligned} \frac{\partial \mathcal{E}}{\partial w_{i21}} &= \frac{-2}{N} \sum_{n=0}^{N-1} (y[n] - \hat{y}[n]) \frac{\partial \hat{y}[n]}{\partial z[n]} \frac{\partial z[n]}{\partial v_1[n]} \frac{\partial v_1[n]}{\partial u_1[n]} \frac{\partial u_1[n]}{\partial w_{i21}} \\ &= \frac{-2}{N} \sum_{n=0}^{N-1} e[n] (1 - \hat{y}[n]^2) w_{o1} (1 - v_1[n]^2) x_2[n] \end{aligned}$$

Training MLPs

Example: XOR problem

Binary classification problem (2D data with 500 pts/class)

- Gradient for output weights $\frac{\partial \mathcal{E}}{\partial \mathbf{w}_o} = -\frac{2}{N} \sum_n e[n] (1 - \hat{y}[n]^2) \mathbf{v}[n]$
- Gradient for input biases $\frac{\partial \mathcal{E}}{\partial \mathbf{b}} = -\frac{2}{N} \sum_n e[n] (1 - \hat{y}[n]^2) \mathbf{w}_o (1 - \mathbf{v}[n]^2)$
- Gradient for input weights $\mathbf{W}_i$:

$$\begin{aligned} \frac{\partial \mathcal{E}}{\partial w_{i12}} &= \frac{-2}{N} \sum_{n=0}^{N-1} (y[n] - \hat{y}[n]) \frac{\partial \hat{y}[n]}{\partial z[n]} \frac{\partial z[n]}{\partial v_2[n]} \frac{\partial v_2[n]}{\partial u_2[n]} \frac{\partial u_2[n]}{\partial w_{i12}} \\ &= \frac{-2}{N} \sum_{n=0}^{N-1} e[n] (1 - \hat{y}[n]^2) w_{o2} (1 - v_2[n]^2) x_1[n] \end{aligned}$$

$$\begin{aligned} \frac{\partial \mathcal{E}}{\partial w_{i22}} &= \frac{-2}{N} \sum_{n=0}^{N-1} (y[n] - \hat{y}[n]) \frac{\partial \hat{y}[n]}{\partial z[n]} \frac{\partial z[n]}{\partial v_2[n]} \frac{\partial v_2[n]}{\partial u_2[n]} \frac{\partial u_2[n]}{\partial w_{i22}} \\ &= \frac{-2}{N} \sum_{n=0}^{N-1} e[n] (1 - \hat{y}[n]^2) w_{o2} (1 - v_2[n]^2) x_2[n] \end{aligned}$$

Training MLPs

Example: XOR problem

Binary classification problem (2D data with 500 pts/class)

- Gradient for output weights $\frac{\partial \mathcal{E}}{\partial \mathbf{w}_o} = -\frac{2}{N} \sum_n e[n] (1 - \hat{y}[n]^2) \mathbf{v}[n]$
- Gradient for input biases $\frac{\partial \mathcal{E}}{\partial \mathbf{b}} = -\frac{2}{N} \sum_n e[n] (1 - \hat{y}[n]^2) \mathbf{w}_o (1 - \mathbf{v}[n]^2)$
- Gradient for input weights $\frac{\partial \mathcal{E}}{\partial \mathbf{W}_i} = -\frac{2}{N} \sum_n e[n] (1 - \hat{y}[n]^2) \mathbf{x}[n] (\mathbf{w}_o (1 - \mathbf{v}[n]^2))^T$

PYTHON PSEUDO-CODE:

Consider that the data is in a Numpy array, $\mathbf{X}$, of dimensions $2 \times N$, and that the desired output (class labels) are store in an array $\mathbf{y}$ of N elements. Furthermore the network weights $\mathbf{w}_o$, $\mathbf{b}$, $\mathbf{W}_i$ are in arrays $\mathbf{w}_o, \mathbf{b}, \mathbf{W}_i$ of dimensions 2×1 , 2×1 and 2×2 , respectively.

- Forward Pass:
 - $\mathbf{U} = \mathbf{np.dot(W_i.T, X) + b}$ $2 \times N$ matrix
 - $\mathbf{V} = \mathbf{np.tanh(U)}$ $2 \times N$ matrix
 - $\mathbf{Z} = \mathbf{np.dot(w_o.T, V)}$ $1 \times N$ matrix
 - $\mathbf{Y_h} = \mathbf{np.tanh(Z)}$ $1 \times N$ matrix
- Error: $\mathbf{E} = \mathbf{Y - Y_h}$ $1 \times N$ matrix

Training MLPs

Example: XOR problem

Binary classification problem (2D data with 500 pts/class)

- Gradient for output weights $\frac{\partial \mathcal{E}}{\partial \mathbf{w}_o} = -\frac{2}{N} \sum_n e[n] (1 - \hat{y}[n]^2) \mathbf{v}[n]$
- Gradient for input biases $\frac{\partial \mathcal{E}}{\partial \mathbf{b}} = -\frac{2}{N} \sum_n e[n] (1 - \hat{y}[n]^2) \mathbf{w}_o (1 - \mathbf{v}[n]^2)$
- Gradient for input weights $\frac{\partial \mathcal{E}}{\partial \mathbf{W}_i} = -\frac{2}{N} \sum_n e[n] (1 - \hat{y}[n]^2) \mathbf{x}[n] (\mathbf{w}_o (1 - \mathbf{v}[n]^2))^T$

PYTHON PSEUDO-CODE:

Consider that the data is in a Numpy array, $\mathbf{X}$, of dimensions $2 \times N$, and that the desired output (class labels) are store in an array $\mathbf{y}$ of N elements. Furthermore the network weights $\mathbf{w}_o$, $\mathbf{b}$, $\mathbf{W}_i$ are in arrays $\mathbf{w}_o, \mathbf{b}, \mathbf{W}_i$ of dimensions 2×1 , 2×1 and 2×2 , respectively.

- Gradients:

<code>dz=-2*E*(1-Yh**2)</code>	$1 \times N$ matrix
<code>dwo=np.dot(V,dz.T)</code>	2×1 matrix
<code>dgu=(1-V**2)*np.dot(wo,dz)</code>	$2 \times N$ matrix
<code>db=np.sum(dgu,axis=1)</code>	2×1 matrix
<code>dWi=np.dot(X,dgu.T)</code>	2×2 matrix

Training MLPs

Example: XOR problem

Binary classification problem (2D data with 500 pts/class)

- Gradient for output weights $\frac{\partial \mathcal{E}}{\partial \mathbf{w}_o} = -\frac{2}{N} \sum_n e[n] (1 - \hat{y}[n]^2) \mathbf{v}[n]$
- Gradient for input biases $\frac{\partial \mathcal{E}}{\partial \mathbf{b}} = -\frac{2}{N} \sum_n e[n] (1 - \hat{y}[n]^2) \mathbf{w}_o (1 - \mathbf{v}[n]^2)$
- Gradient for input weights $\frac{\partial \mathcal{E}}{\partial \mathbf{W}_i} = -\frac{2}{N} \sum_n e[n] (1 - \hat{y}[n]^2) \mathbf{x}[n] (\mathbf{w}_o (1 - \mathbf{v}[n]^2))^T$

PYTHON PSEUDO-CODE:

Consider that the data is in a Numpy array, $\mathbf{X}$, of dimensions $2 \times N$, and that the desired output (class labels) are store in an array y of N elements. Furthermore the network weights $\mathbf{w}_o$, $\mathbf{b}$, $\mathbf{W}_i$ are in arrays $\mathbf{w}_o, \mathbf{b}, \mathbf{W}_i$ of dimensions 2×1 , 2×1 and 2×2 , respectively.

- Adaptations:

$\mathbf{w}_o = \mathbf{w}_o - \eta \mathbf{d}\mathbf{w}_o$	2×1 matrix
$\mathbf{b} = \mathbf{b} - \eta \mathbf{d}\mathbf{b}$	2×1 matrix
$\mathbf{W}_i = \mathbf{W}_i - \eta \mathbf{d}\mathbf{W}_i$	2×2 matrix

Training MLPs

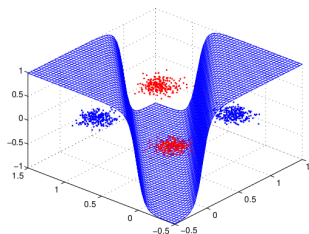
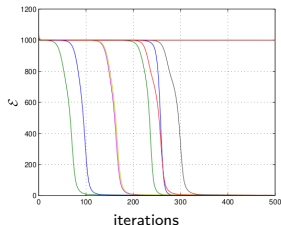
Example: XOR problem

Binary classification problem (2D data with 500 pts/class)

- Gradient for output weights $\frac{\partial \mathcal{E}}{\partial \mathbf{w}_o} = -\frac{2}{N} \sum_n e[n] (1 - \hat{y}[n]^2) \mathbf{v}[n]$
- Gradient for input biases $\frac{\partial \mathcal{E}}{\partial \mathbf{b}} = -\frac{2}{N} \sum_n e[n] (1 - \hat{y}[n]^2) \mathbf{w}_o (1 - \mathbf{v}[n]^2)$
- Gradient for input weights $\frac{\partial \mathcal{E}}{\partial \mathbf{W}_i} = -\frac{2}{N} \sum_n e[n] (1 - \hat{y}[n]^2) \mathbf{x}[n] (\mathbf{w}_o (1 - \mathbf{v}[n]^2))^T$

RESULTS:

- Different initialization lead to different results.
- Important to visualize the error.



NN discriminant function

Training MLPs

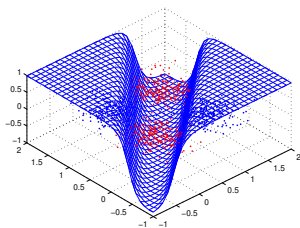
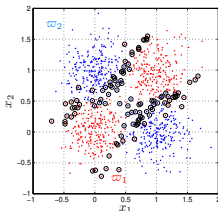
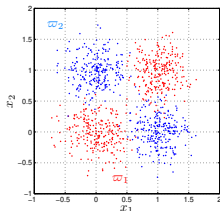
Example: XOR problem

Binary classification problem (2D data with 500 pts/class)

- Gradient for output weights $\frac{\partial \mathcal{E}}{\partial \mathbf{w}_o} = -\frac{2}{N} \sum_n e[n] (1 - \hat{y}[n]^2) \mathbf{v}[n]$
- Gradient for input biases $\frac{\partial \mathcal{E}}{\partial \mathbf{b}} = -\frac{2}{N} \sum_n e[n] (1 - \hat{y}[n]^2) \mathbf{w}_o (1 - \mathbf{v}[n]^2)$
- Gradient for input weights $\frac{\partial \mathcal{E}}{\partial \mathbf{W}_i} = -\frac{2}{N} \sum_n e[n] (1 - \hat{y}[n]^2) \mathbf{x}[n] (\mathbf{w}_o (1 - \mathbf{v}[n]^2))^T$

RESULTS:

- Noisy data. 2 hidden units MLP network.



Training MLPs

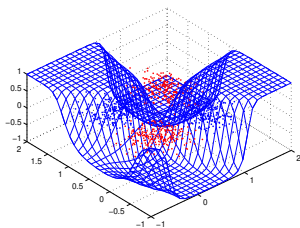
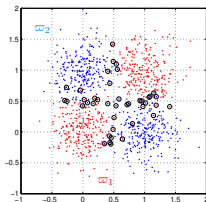
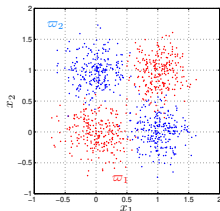
Example: XOR problem

Binary classification problem (2D data with 500 pts/class)

- Gradient for output weights $\frac{\partial \mathcal{E}}{\partial \mathbf{w}_o} = -\frac{2}{N} \sum_n e[n] (1 - \hat{y}[n]^2) \mathbf{v}[n]$
- Gradient for input biases $\frac{\partial \mathcal{E}}{\partial \mathbf{b}} = -\frac{2}{N} \sum_n e[n] (1 - \hat{y}[n]^2) \mathbf{w}_o (1 - \mathbf{v}[n]^2)$
- Gradient for input weights $\frac{\partial \mathcal{E}}{\partial \mathbf{W}_i} = -\frac{2}{N} \sum_n e[n] (1 - \hat{y}[n]^2) \mathbf{x}[n] (\mathbf{w}_o (1 - \mathbf{v}[n]^2))^T$

RESULTS:

- Noisy data. 5 hidden units MLP network.



Training MLPs

Extra Considerations

In training MLPs there several points one needs to take in consideration. So far, we have seen a simple MLP, on hidden layer network, trained by minimizing the Mean Squared Error through steepest descend. The state of the art has evolved to a point where now there are many decisions to make when training a NN.

- There are many error functions one can choose from. The same also applies to activation functions, optimizations methods and weight initializations.
- Another important and often overlooked aspect is data normalization. Most of the times, it is essential for a successful training process to normalize the data (there are many ways to do this in `sklearn` and `tensorflow`).
- One more factor to take in account when adapting a MLP is to avoid over-fitting the training data. MLPs can easily do this through ℓ_1 and ℓ_2 regularization, but many other techniques should also be considered. such as *Early Stopping*, *Batch Normalization*, *Dropout Layer* and *Data Augmentation* (these will be covered later on in the course).
- Finally, examine our training and testing methodology. Note that for large networks and datasets, sometimes one has to revert to simple train/test schemes (or train/validation/test split of the data), since it is not possible to implement cross-validation or other more computational demanding approaches.

Neural Networks with Tensorflow-Keras

NN with Tensorflow-Keras

- Tensorflow is an open source machine learning library developed by Google Brain Team, specially suited for deep neural networks. First released in 2015, it has since then undergone major changes, namely incorporating the Keras high level API.

Check out:

- [TensorFlow](#)

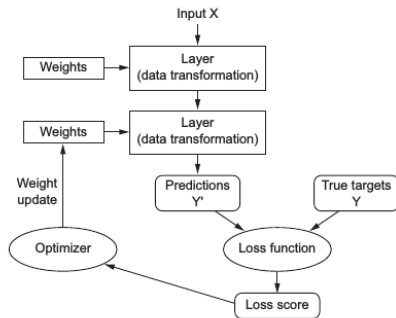
- [Keras](#)

- Online courses:
 - [Deep Learning AI](#)
 - [Coursera](#)
 - [Udacity](#)
 - [Stanford Online](#)
 - [PyImageSearch](#)
 - ...

NN with Tensorflow-Keras

Constructing a neural network with `tf-keras` revolves around the following programming blocks:

- Defining the *layers* and combining them into a *network* (or *model*).
- Choosing a *cost* function to be minimized by the network
- Choosing an *optimizer* for the weight adaptation.
- Training the network.



by François Chollet

NOTE: there are other processing blocks in machine learning project using neural networks, namely the data preparation, the evaluation, deploying and monitoring the developed system .

NN with Tensorflow-Keras

Building Neural Networks:

A *fully connected* neural network is a stack of layer. First one needs to create a model, then define the layers and their number of units and their *activation functions*, and finally compile the .

Here is an example of the TF-Keras implementation of a 3 layer MLP network, for 20-dimensional input data:

```
import tensorflow as tf
import tensorflow.keras as keras
nn=keras.Sequential() or: nn=keras.models.Sequential()
nn.add(keras.Input(shape=(20,)))
nn.add(keras.layers.Dense(units=32,activation="tanh"))
nn.add(keras.layers.Dense(10,activation="softmax"))
nn.compile(optimizer="adam", \
loss="categorical_crossentropy",metrics=["accuracy"])
```

NOTE: The above code creates a model by adding two layers with the function `.add()`. For the first layer, the input data shape has to be specified. Like in many other cases in Python, this is not the only way to define a model. For example, the next lines of code define the same model as the above.

```
nn=keras.Sequential([keras.layers.Dense(32,input_shape=X.shape[0], \
activation="tanh") \
keras.layers.Dense(10,activation="softmax")])
```

NN with Tensorflow-Keras

Building Neural Networks:

A *fully connected* neural network is a stack of layer. First one needs to create a model, then define the layers and their number of units and their *activation functions*, and finally compile the .

Here is an example of the TF-Keras implementation of a 3 layer MLP network, for 20-dimensional input data:

```
import tensorflow as tf
import tensorflow.keras as keras
nn=keras.Sequential() or: nn=keras.models.Sequential()
nn.add(keras.Input(shape=(20,)))
nn.add(keras.layers.Dense(units=32,activation="tanh"))
nn.add(keras.layers.Dense(10,activation="softmax"))
nn.compile(optimizer="adam", \
loss="categorical_crossentropy",metrics=["accuracy"])
```

NOTE: To check the configuration of a given model, use the `.summary()` method:

```
nn.summary()
```

Layer (type)	Output Shape	Param #
--------------	--------------	---------

dense_1 (Dense)	(None, 32)	672
-----------------	------------	-----

dense_2 (Dense)	(None, 10)	330
-----------------	------------	-----

Total params: 1,002		
---------------------	--	--

NN with Tensorflow-Keras

Building Neural Networks:

A *fully connected* neural network is a stack of layer. First one needs to create a model, then define the layers and their number of units and their *activation functions*, and finally compile the .

Here is an example of the TF-Keras implementation of a 3 layer MLP network, for 20-dimensional input data:

```
import tensorflow as tf
import tensorflow.keras as keras
nn=keras.Sequential() or: nn=keras.models.Sequential()
nn.add(keras.Input(shape=(20,)))
nn.add(keras.layers.Dense(units=32,activation="tanh"))
nn.add(keras.layers.Dense(10,activation="softmax"))
nn.compile(optimizer="adam", \
loss="categorical_crossentropy",metrics=["accuracy"])
```

Models:

The first step is to create a model. Keras has available two types of models: the *sequential models* and models defined with *Keras functional API*.

- Sequential models consist in a stack of layers. This model is enough for most applications and will be the preferred choice in this course.
- The Keras functional API allows to define more complex models, such as multi-output models, directed acyclic graphs, or models with shared layers.

NN with Tensorflow-Keras

Building Neural Networks - Layers:

- The previous code is a stack of 3 Dense layers. Dense layers are fully connected layers, in the process of building neural network, one has to decide **how many layers** to use and **how many units** in each layer.
- The more layers, and the more units per layer one uses, the more the network is capable of discriminating complex problems, but at the same time, the networks become more susceptible of **overfitting** the data.
- Dense layers perform a linear (affine) transformation followed by a non-linearity (the activation function): $\mathbf{z} = \varphi(\mathbf{W}\mathbf{v} + \mathbf{b})$.
Without the activation functions, a stack of Dense layers would simply result in a linear transformation: $\hat{\mathbf{y}} = \mathbf{W}\mathbf{x} + \mathbf{b}$

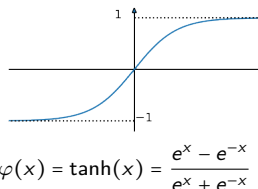
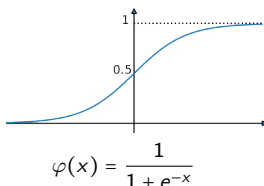
There are many types of layers in tf-keras, some used for pre-processing (e.g Flatten, Masking), others used for convolutional and recurrent networks, and others used for regularization purposes.

In the process of defining a layer, and once the number of units is set, there are still many decisions to make, such as weights and biases initializations and regularizations, and the choice of activation functions.

NN with Tensorflow-Keras

Activation Functions:

- The activation functions initially used in MLP networks were the sigmoid and the hyperbolic tangent. These functions are saturating functions (inspired from biological perspective).



- These functions have severe limitations in terms of adapting the weights of a MLP network with more than 3 layers. The problem is *the vanishing gradient*. The backpropagation algorithm works from the output layer to the input layer, propagating the error gradients. Since the gradients of these activation functions are close to zero for most of their domain, the gradients often get smaller and smaller as the algorithm progresses down the lower layers, leaving the lower layers connection weights virtually unchanged. In these situations, the training never converges to a good solution.

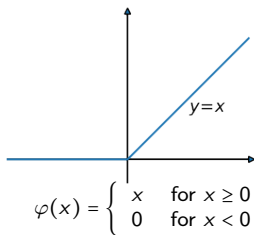
NN with Tensorflow-Keras

Activation Functions:

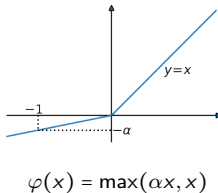
- The vanishing gradient problem is solved by using *non-saturating* activations functions. Among these, the most common are:

ReLU:

Rectified Linear Unit

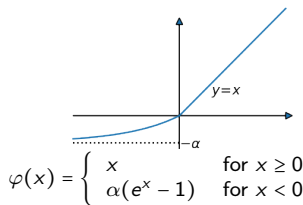


Leaky ReLU:



ELU:

Exponential Linear Unit



NN with Tensorflow-Keras

Activation Functions:

- There are other activation functions available under the module `keras.activations`. These include the `linear` activation (*i.e* identity transformation - used in regression), `softsign`, `hard_sigmoid`, scaled ELU and many others. The activations can be specified when creating the layers. For example, the next command adds a Dense layer with ReLU activations:

```
nn.add(keras.layers.Dense(32,activation="relu"))
```

or

```
nn.add(keras.layers.Dense(32,activation=keras.activations.relu))
```

- There is also a special activation function used in the last layer of a MLP for multi-class classification problem: the **softmax** activation. The softmax converts the outputs of the network into class probabilities. Given a network with c outputs, $\hat{y}_i$ with $i = 1, \dots, c$, the new (softmax) outputs are:

$$\hat{z}_i = \frac{e^{\hat{y}_i}}{\sum_j e^{\hat{y}_j}}$$

After the applying the softmax normalization the outputs, $\hat{z}_i$ will be in the interval $[0, 1]$ and $\sum_i \hat{z}_i = 1$.

NN with Tensorflow-Keras

Regularization Techniques:

Regularization refers to methods that prevent the networks to overfit the training data.

- A standard approach is to apply a penalty to the network weights (usually not the biases). The available regularizations in tf-keras are ℓ_1 , ℓ_2 , and a combination of both. The type of regularization is specified in the layer command:

```
keras.layers.Dense(32,activation="relu",\nkernel_regularizer=keras.regularizer.l1(0.02))
```
- Another way to avoid the network from overfitting is to use **Dropout**. Dropout is a very popular regularization technique for deep learning networks (by G. Hinton 2012). In this approach, each units weights have a probability of being temporarily “dropped out” during each training step. In tf-keras this is done by using a Dropout layer after each Dense layer. The next commands add a dropout layer (with $p=0.5$) to dense layer. Note that dropout layers do not have any weights (since they are not part of the network) .

```
nn.add(keras.layers.Dense(32,activation="relu"))\nn.add(keras.layers.Dropout(0.5))
```
- Another technique to avoid overfitting is to stop the training is the error in the validation set starts to increase. This is known as **early stopping**. This can be easily implemented by saving the weights of the network every time the best performance on the validation set is attained, and use the winning model.

NN with Tensorflow-Keras

Compiling the Model:

Once the model defined, it needs to be compiled. This means choosing a loss function and an optimization method.

- **Loss Functions:**

Loss functions are inversely proportional to the performance of the network. The functions are dependent on the outputs of the network, and they are to be minimized during the training by adjusting the weights and biases of a network. There are many loss functions (see [Keras Losses](#)) such as MSE, MAE, hinge, etc. For classification, the following are usually a good choice:

- ▶ Binary cross-entropy for two class problems (binary classification)
`nn.compile(loss="binary_crossentropy")`
- ▶ Categorical cross entropy for multi-class classification
`nn.compile(loss="categorical_crossentropy")`

- **Optimizers:**

There are many optimization techniques available in Keras (see [Keras Optimizers](#)). These are based on ameliorations of the maximum gradient descent procedure. These include the stochastic gradient descent (SGD), the Nesterov, the Adam, the Adagrad and variants, RMSProp and others. ex.: `nn.compile(optimizer="Adam")`

NN with Tensorflow-Keras

Training the Model:

Once compiled, the model is ready to be trained. This is accomplished using the function `fit()`. There are several important parameters to take in account:

- Input dataset and labels.
- Validation set and labels.

It is important to use a validation set to assess the performance of the network by visualizing the training/validation loss/accuracy evolution. The validation set also enables the use of early stopping. Training methodologies should, if possible, include cross-validation or shuffle/split approaches.

- Number of epochs.

An epoch is a pass over the entire training set during training. The division of the training into epochs is useful for periodic evaluation and logging purposes. Keras also has the possibility to run **callbacks** at the end of each epoch, like learning rate changes or model saving.

- Batch size.

The variable `batch_size` specifies the number of training samples used in each weight adaptation. Larger batch sizes usually result in better approximations at the expense of a longer training time.

Ex: training the previous network for 10 epochs:

```
train=nn.fit(Xtrain,ytrain,epochs=10,batch_size=1024,\nvalidation_data=(Xvalid,yvalid))
```

NN with Tensorflow-Keras

Training the Model:

Once compiled, the model is ready to be trained. This is accomplished using the function `fit()`. There are several important parameters to take in account:

- Input dataset and labels.
- Validation set and labels.
It is important to use a validation set to assess the performance of the network by visualizing the training/validation loss/accuracy evolution. The validation set also enables the use of early stopping. Training methodologies should, if possible, include cross-validation or shuffle/split approaches.
- Number of epochs.
An epoch is a pass over the entire training set during training. The division of the training into epochs is useful for periodic evaluation and logging purposes. Keras also has the possibility to run **callbacks** at the end of each epoch, like learning rate changes or model saving.
- Batch size.
The variable `batch_size` specifies the number of training samples used in each weight adaptation. Larger batch sizes usually result in better approximations at the expense of a longer training time.

Ex: training the previous network for 10 epochs:

It is important to specify a return variable to see the accuracy and loss evolutions during training.

NN with Tensorflow-Keras

MNIST Dataset:

Multi-class problem, with 10 classes (images of handwritten digits 0-9).

Training set: 60 000 images

Test set: 10 000 images



Objectives:

- Train a MLP network with this data.
- Use a validation set to evaluate the performance.
- Assess the quality of the network and spot any signs of overfitting.

NN with Tensorflow-Keras

MNIST Dataset:

Load needed Python modules, the MNIST dataset and prepare it for classification.

- Python modules:

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow.keras as keras
from tensorflow.keras.datasets import mnist
```

- Load data and divide the training data into train and validation sets.

```
(Xtrain,ytrain),(Xtest,ytest)=mnist.load_data()
Ntrain=40000
Xvalid=Xtrain[Ntrain:]*1. # "*1."→ to convert to float
yvalid=ytrain[Ntrain:]
Xtrain=Xtrain[:Ntrain]*1.
ytrain=ytrain[:Ntrain]
Xtest=Xtest*1.
(suggestion: use the StandardScaler from sklearn to normalize the data).
```

- Convert labels into vectors (one hot encoding).

```
ytrainB=keras.utils.to_categorical(ytrain)
yvalidB=keras.utils.to_categorical(yvalid)
ytestB=keras.utils.to_categorical(ytest)
```

NN with Tensorflow-Keras

MNIST Dataset:

Build, compile and train a fully connected 3 layer MLP network, with 50 units in the hidden layer. Although the input data consists of images, in this approach (as in most of ML cases), the data is first converted into vector format (with 784 dimensions). Note that the same conversion process has to be done to the test data or to new data fed into the network.

- Defining the 3-layers MLP network:

```
nn=keras.Sequential()  
nn.add(keras.layers.Flatten(input_shape=[28,28]))  
nn.add(keras.layers.Dense(50, activation="tanh"))  
nn.add(keras.layers.Dense(10,activation='softmax'))
```

NOTE: 1) The first layer is just a reshape of the inputs (it does not have any parameters).

This could also have been done in the pre-processing phase of the data.

2) The last layers is a softmax activation (outputs = number of classes).

- Compiling the model

```
nn.compile(optimizer="nadam",\  
loss="categorical_crossentropy",metrics=["accuracy"])  
(suggestion: try different optimizers).
```

NN with Tensorflow-Keras

MNIST Dataset:

Training the model.

- Training the model for 10 epochs (should be longer).

```
train=nn.fit(Xtrain, ytrainB, epochs=10\  
            ,batch_size=1024,validation_data=(Xvalid, yvalidB))
```

Train on 40000 samples, validate on 20000 samples

Epoch 1/10

40000/40000 [=====] - 1s 16us/sample - loss: 1.2974 - acc: 0.5868 - val_loss: 0.7243 - val_acc: 0.7901

Epoch 2/10

40000/40000 [=====] - 0s 12us/sample - loss: 0.5901 - acc: 0.8301 - val_loss: 0.5206 - val_acc: 0.8492

Epoch 3/10

40000/40000 [=====] - 0s 12us/sample - loss: 0.4705 - acc: 0.8640 - val_loss: 0.4453 - val_acc: 0.8672

:

Epoch 10/10 40000/40000 [=====] - 0s 12us/sample - loss: 0.3213 - acc: 0.9043 - val_loss: 0.3271 - val_acc: 0.9040

NN with Tensorflow-Keras

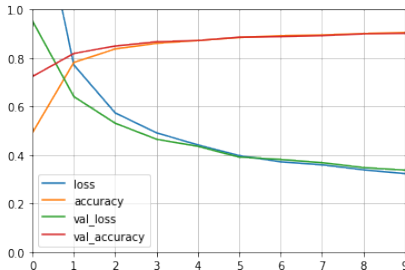
MNIST Dataset:

Training the model.

- Training the model for 10 epochs (should be longer).

```
train=nn.fit(Xtrain, ytrainB, epochs=10\
             ,batch_size=1024,validation_data=(Xvalid, yvalidB))
```
- Visualizing the loss and accuracy evolution during training (final accuracy $\approx 90\%$)

```
h=train.history
plt.plot(h["loss"]),plt.plot(h["acc"])
plt.plot(h["val_loss"]),plt.plot(h["val_acc"])
```



NN with Tensorflow-Keras

MNIST Dataset:

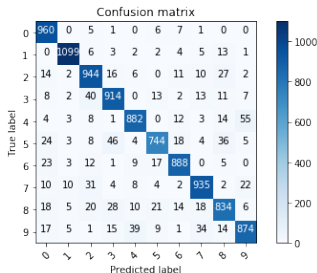
Making predictions and test-set evaluation:

- Predictions on the test set.
The predicted outputs have to be converted back to integer labels (0-9).

```
yOut=nn.predict(Xtest)
ytestPred=np.argmax(yOut,axis=1)
```

- Print the confusion matrix.

```
from sklearn.metrics import confusion_matrix
print(confusion_matrix(ytest,ytestPred))
```



- Accuracy on the test set (`nn.evaluate(Xtest,ytestB)`)
10000/10000 [=====] - 0s 29us/sample - loss: 0.3106 - acc: 0.9074

Saving Models: Recommended Practice

Recommended format

Use `.keras` to save a complete Keras model.

- Stores:
 - architecture
 - weights
 - compile configuration
 - optimizer state
- Best choice for saving and later reloading in Python.
- Check out the [Keras Documentation](#) on how to save models.

Example

```
model.save("model.keras")
```

Load later

```
from tensorflow import keras  
model = keras.models.load_model("model.keras")
```

Saving Weights Only

Weights-only workflow

Saving only weights is still a standard and useful practice.

- Useful for:
 - transfer learning
 - reusing a known architecture
 - lighter checkpoints
- A common filename convention is: `model.weights.h5`
Keras uses the Python package `h5py` which is a interface for the binary data format HDF5 (see www.h5py.org).
If you need to install the package: `pip -install h5py`.

Save weights

```
model.save_weights("model.weights.h5")
```

Load weights

```
model.load_weights("model.weights.h5")
```

Saving Weights Only

Weights-only workflow

Saving only weights is still a standard and useful practice.

- Useful for:
 - transfer learning
 - reusing a known architecture
 - lighter checkpoints
- A common filename convention is: `model.weights.h5`
Keras uses the Python package `h5py` which is a interface for the binary data format HDF5 (see www.h5py.org).
If you need to install the package: `pip -install h5py`.

Important

When loading only weights, the model architecture must already be defined.

```
model = keras.Sequential()  
model.add(keras.layers.Input(shape=(784,))  
model.add(keras.layers.Dense(128, activation="relu"))  
model.add(keras.layers.Dense(10, activation="softmax"))  
model.load_weights("model.weights.h5")
```

NN with Tensorflow-Keras

Callbacks:

Callbacks are functions that can be applied during the training stage. Callbacks have access to all the available data about the state of the model and its performance, and they can be used for various actions. Here are a few of its usages:

- **Logging** training and validation data during training. Among the the callbacks used for this purpose are BaseLogger (automatically applied to Keras models), the progbarLogger the CSVLogger, and the History callback (also automatically applied) .
- Creating **model checkpoints** by saving the model after every epoch:
`tensorflow.keras.callbacks.ModelCheckpoint(filepath="best_model.keras", \monitor="val_loss", verbose=1, save_best_only=True, \save_weights_only=False, mode="min", save_freq="epoch")`
- **Early stopping** the training process. This callback is used to stop the training when the performance has stopped improving:
`tensorflow.keras.callbacks.EarlyStopping(monitor="val_accuracy", \save_best_only=True, verbose=1, mode="max", restore_best_weights=True)`
- There are many callbacks and custom callbacks can be also created. Check out the [Keras Documentation](#).

NN with Tensorflow-Keras

Callbacks:

Callbacks are passed as a list (with the keyword `callbacks`) to the `.fit()` function of the model. The following code trains the model of the previous example with three callbacks:

```
from tensorflow.keras import callbacks
CB1=callbacks.ModelCheckpoint(filepath="best_nn.keras", save_best_only=True)
CB2=callbacks.EarlyStopping(monitor="val_loss",patience=5, \
verbose=1, mode="auto", restore_best_weights=True)
CB3=callbacks.CSVLogger("nn_log.csv", separator=";", append=False)
CBlist=[CB1,CB2,CB3]

train=nn.fit(..., callbacks=CBlist)
```